

UniApp Android 原生插件开发全解析

UniApp Android 原生插件开发全解析

UniApp 原生插件开发（Android 端）是为了弥补跨端框架在原生能力调用上的不足，通过编写 Android 原生代码扩展 UniApp 功能，比如调用硬件接口、集成第三方 SDK、优化性能等。以下从开发环境搭建、插件核心结构、开发流程、调试与打包、常见问题等维度详细讲解。

一、开发环境准备

1. 基础环境

- **UniApp 开发端**：HBuilderX（最新版，需安装“App 开发”插件）。
- **Android 开发端**：
 - JDK 8（UniApp 对高版本 JDK 兼容性差，优先 1.8）；
 - Android Studio（2020.3.1 及以上，需配置 SDK：建议 API 21+，对应 Android 5.0，覆盖绝大多数设备）；
 - Gradle 版本与 Android Studio 适配（如 7.0+）。
- **设备/模拟器**：Android 真机（开启开发者模式、USB 调试）或 Android Studio 模拟器（建议 API 24+）。

2. 环境配置关键步骤

- 配置 `ANDROID_HOME` 环境变量（指向 Android SDK 安装路径）；
- 配置 JDK 环境变量（`JAVA_HOME` 指向 JDK 8 路径）；
- HBuilderX 中配置“运行配置”：设置 Android 真机/模拟器连接（HBuilderX 会自动识别已连接的设备）。

二、UniApp Android 原生插件核心概念

1. 插件类型

UniApp Android 原生插件分两种核心类型，适配不同场景：

类型	特点	适用场景
模块插件（Module）	无 UI，纯逻辑调用（如网络请求、硬件接口、SDK 集	工具类功能（如支付、定位、蓝牙）

	成)，通过 <code>uni.requireNativePlugin</code> in 调用	
组件插件（Component）	带 UI 界面（如自定义按钮、地图组件），可在 UniApp 页面中像普通组件一样使用	自定义可视化控件

2. 核心交互逻辑

UniApp 与 Android 原生通过 **JSBridge** 通信：

- JS 端（UniApp）调用原生方法 → 原生处理逻辑 → 回调结果给 JS 端；
- 原生主动触发事件 → JS 端监听并处理（如硬件状态变化回调）。
- 数据传递：支持基础类型（int、String、boolean）、JSON 对象、数组，复杂数据需通过 JSON 序列化/反序列化。

3. 插件目录结构（标准）

UniApp 原生插件最终需打包为 `.aar` 文件，开发阶段的 Android 工程结构如下：

Code block

```
1  |—— app                                // 测试模块（可选，用于原生侧调试）
2  |—— pluginName                        // 插件核心模块（必须，名称自定义）
3  |   |—— src
4  |   |   |—— main
5  |   |   |   |—— java
6  |   |   |   |   |—— com/xxx/plugin // 包名（自定义，建议反向域名）
7  |   |   |   |   |   |—— Module.java // 模块插件核心类（继承 UniModule）
8  |   |   |   |   |   |—— Component.java // 组件插件核心类（继承 UniComponent）
9  |   |   |   |—— res                    // 资源文件（布局、图片、字符串等）
10 |   |   |   |—— AndroidManifest.xml // 插件清单（权限、组件声明）
11 |   |   |—— build.gradle                // 模块构建配置
12 |—— build.gradle                        // 工程根构建文件
13 |—— settings.gradle                    // 模块配置
14 |—— uniplugin.json                     // 插件配置（关键！UniApp 识别插件的核心）
```

三、模块插件（无 UI）开发全流程

以“自定义 Toast 提示”为例，开发一个简单的模块插件：

步骤 1：创建 Android 插件工程

1. 打开 Android Studio，新建“Empty Project”，命名为 `UniPluginDemo`；

2. 新建 Module：选择“Android Library”，命名为 `toastPlugin`（插件名），包名建议 `com.xxx.toast`；

3. 引入 UniApp 原生插件依赖：

- 在 `toastPlugin` 的 `build.gradle` 中添加依赖（UniApp 提供的核心库）：

Code block

```
1 dependencies {
2     // UniApp 核心依赖（必须，版本与 HBuilderX 适配）
3     implementation 'com.github.dcloudio:uniplugin-common:2.0.0'
4     // 若需兼容低版本，可添加
5     implementation 'androidx.appcompat:appcompat:1.4.1'
6 }
```

- 注意：需确保 `build.gradle` 中 `compileSdkVersion` ≥ 21 ，`minSdkVersion` ≥ 21 。

步骤 2：编写插件核心类（继承 UniModule）

创建 `ToastModule.java`，继承 `UniModule`（UniApp 封装的模块基类），核心代码：

Code block

```
1 package com.xxx.toast;
2
3 import android.content.Context;
4 import android.widget.Toast;
5 import com.taobao.weex.annotation.JSMethod;
6 import com.taobao.weex.bridge.JSCallback;
7 import com.xxx.uniplugin.UniModule;
8
9 // 必须添加 @UniJSMethod 注解的方法才会被 JS 端调用
10 public class ToastModule extends UniModule {
11
12     // 同步方法：显示短 Toast
13     @JSMethod(uiThread = true) // uiThread=true: 在主线程执行（UI 操作必须主线程）
14     public void showShortToast(String msg) {
15         Context context = mWXSDKInstance.getContext(); // 获取 UniApp 上下文
16         Toast.makeText(context, msg == null ? "默认提示" : msg,
17             Toast.LENGTH_SHORT).show();
18     }
19
20     // 异步方法：显示长 Toast，并回调结果给 JS
21     @JSMethod(uiThread = true)
22     public void showLongToast(String msg, JSCallback callback) {
```

```

22         Context context = mWXSDKInstance.getContext();
23         Toast.makeText(context, msg, Toast.LENGTH_LONG).show();
24         // 回调结果 (JSON 格式)
25         if (callback != null) {
26             HashMap<String, Object> result = new HashMap<>();
27             result.put("code", 0);
28             result.put("msg", "Toast 显示成功");
29             callback.invoke(result); // 单次回调
30             // callback.invokeAndKeepAlive(result); // 保持回调 (用于持续返回数据)
31         }
32     }
33 }

```

关键注解说明：

- `@JSMethod`：标记可被 JS 调用的方法，必填；
- `UiThread = true`：方法在 Android 主线程执行（UI 操作必须设为 true）；`false` 则在子线程执行（适合耗时操作）。

步骤 3：配置 uniplugin.json（插件声明）

在插件模块的 `src/main/assets` 目录下新建 `uniplugin.json`（若没有 `assets` 目录则创建），内容如下（UniApp 识别插件的核心配置）：

Code block

```

1  {
2      "plugins": [
3          {
4              "type": "module", // 插件类型: module (模块) / component (组件)
5              "name": "ToastPlugin", // 插件名称 (JS 端调用时的标识)
6              "class": "com.xxx.toast.ToastModule" // 插件核心类的完整路径
7          }
8      ]
9  }

```

步骤 4：打包插件为 AAR 文件

1. 在 Android Studio 中，点击 `Build → Make Module 'toastPlugin'`，或直接执行 `gradlew assembleRelease`；
2. 打包完成后，AAR 文件路径：`toastPlugin/build/outputs/aar/toastPlugin-release.aar`（需用 release 包，debug 包可能有兼容性问题）。

步骤 5：UniApp 端集成与调用插件

(1) 导入插件

- 在 HBuilderX 中，新建 UniApp 项目（选择“App 项目”）；
- 在项目根目录新建 `nativeplugins` 文件夹 → 新建 `ToastPlugin` 文件夹 → 新建 `android` 文件夹，将上述 `toastPlugin-release.aar` 复制到 `android` 目录；
- 在 `ToastPlugin` 文件夹下新建 `package.json`（插件描述文件），内容：

Code block

```
1  {
2    "name": "ToastPlugin",
3    "id": "ToastPlugin",
4    "version": "1.0.0",
5    "description": "自定义 Toast 插件",
6    "android": {
7      "plugins": [
8        {
9          "type": "module",
10         "name": "ToastPlugin",
11         "class": "com.xxx.toast.ToastModule"
12       }
13     ],
14     "integrateType": "aar"
15   }
16 }
```

(2) JS 端调用插件

在 UniApp 页面（如 `pages/index/index.vue`）中调用：

Code block

```
1  <template>
2    <view>
3      <button @click="showShortToast">显示短 Toast</button>
4      <button @click="showLongToast">显示长 Toast（带回调）</button>
5    </view>
6  </template>
7
8  <script>
9    // 引入原生插件
10   const toastPlugin = uni.requireNativePlugin('ToastPlugin');
11
12   export default {
13     methods: {
14       showShortToast() {
```

```

15      // 调用同步方法
16      toastPlugin.showShortToast('Hello UniApp 原生插件! ');
17  },
18  showLongToast() {
19      // 调用异步方法, 接收回调
20      toastPlugin.showLongToast('异步 Toast 提示', (res) => {
21          console.log('原生回调结果: ', res);
22          uni.showModal({
23              title: '回调结果',
24              content: JSON.stringify(res)
25          });
26      });
27  }
28  }
29  };
30  </script>

```

四、组件插件（带 UI）开发要点

组件插件需继承 `UniComponent`，核心是自定义 View 并挂载到 UniApp 页面，以下是关键差异点：

1. 核心类编写（继承 UniComponent）

Code block

```

1  package com.xxx.button;
2
3  import android.content.Context;
4  import android.view.View;
5  import android.widget.Button;
6  import com.taobao.weex.ui.component.WXComponent;
7  import com.taobao.weex.ui.component.WXComponentProp;
8  import com.taobao.weex.ui.component.WXVContainer;
9  import com.xxx.uniplugin.UniComponent;
10
11  public class CustomButtonComponent extends UniComponent<Button> {
12
13      // 构造方法（必须实现）
14      public CustomButtonComponent(WXSDKInstance instance, WXVContainer parent,
15      BasicComponentData basicComponentData) {
16          super(instance, parent, basicComponentData);
17      }
18
19      // 创建自定义 View
20      @Override

```

```

20     protected Button initViewHostView(Context context) {
21         Button button = new Button(context);
22         button.setTextSize(16);
23         button.setBackgroundColor(0xFF007AFF);
24         // 绑定点击事件
25         button.setOnClickListener(v -> {
26             // 触发 JS 端事件
27             fireEvent("buttonClick", new HashMap<String, Object>(){{
28                 put("msg", "原生按钮被点击");
29             }});
30         });
31         return button;
32     }
33
34     // 接收 JS 端传递的属性 (如 title)
35     @WXComponentProp(name = "btnTitle")
36     public void setBtnTitle(String title) {
37         if (getHostView() != null) {
38             getHostView().setText(title);
39         }
40     }
41 }

```

2. uniplugin.json 配置

Code block

```

1  {
2      "plugins": [
3          {
4              "type": "component",
5              "name": "CustomButton", // 组件名称 (JS 端使用的标签名)
6              "class": "com.xxx.button.CustomButtonComponent"
7          }
8      ]
9  }

```

3. UniApp 端使用组件

Code block

```

1  <template>
2      <view>
3          <!-- 原生组件使用 -->
4          <CustomButton btnTitle="原生按钮" @buttonClick="handleClick"></CustomButton>

```

```
5     </view>
6 </template>
7
8 <script>
9   export default {
10     methods: {
11       handleClick(e) {
12         console.log('原生组件事件: ', e.detail);
13       }
14     }
15   };
16 </script>
```

五、调试与打包

1. 原生侧调试（Android Studio）

- 在 Android Studio 中创建“测试 App 模块”（`app` 模块），引入插件模块依赖；
- 在 `app` 模块中编写测试代码，直接调用插件类的方法，验证逻辑是否正常；
- 运行 `app` 模块到真机/模拟器，调试原生代码（断点、日志输出）。

2. UniApp 侧调试（HBuilderX）

（1）真机调试

- 连接 Android 真机（开启 USB 调试、允许安装未知来源应用）；
- HBuilderX 中点击“运行 → 运行到手机或模拟器 → 选择已连接的真机”；
- 若插件未生效，检查：
 - `package.json` 中插件名称、类路径是否与 `uniplugin.json` 一致；
 - AAR 文件是否为 release 版本；
 - 手机是否安装了旧版本 App（需先卸载）。

（2）日志调试

- 原生侧日志：通过 Android Studio 的 `Logcat` 查看（过滤标签 `UniPlugin`）；
- JS 侧日志：HBuilderX 调试控制台或手机端“调试模式”查看。

3. 插件打包发布

- 若仅自用：将 `nativeplugins` 文件夹随 UniApp 项目一起打包（HBuilderX → 发行 → 原生 App-云打包）；
- 若发布到插件市场：需按 DCloud 规范打包，填写插件信息，上传 AAR 和 `package.json`。

六、常见问题与解决方案

1. 插件调用提示“找不到原生插件”

- 检查 `nativeplugins` 目录结构是否正确（必须是 `nativeplugins/插件名/android/xxx.aar`）；
- 检查 `package.json` 中 `id` 与 JS 端 `requireNativePlugin` 的名称一致；
- HBuilderX 清理缓存（工具 → 清理缓存 → 清理全部）后重新运行。

2. 原生方法不执行/回调无响应

- 确保方法添加了 `@JSMethod` 注解；
- UI 操作需设置 `uiThread = true`；
- 回调对象 `JSCallback` 需判空，避免空指针。

3. 第三方 SDK 集成冲突（如包名、权限）

- 在插件的 `AndroidManifest.xml` 中声明权限（如 `<uses-permission android:name="android.permission.INTERNET"/>`）；
- 若出现类冲突，在 `build.gradle` 中排除冲突依赖：

Code block

```
1  implementation ('com.xxx.sdk:xxx:1.0.0') {  
2      exclude group: 'com.android.support'  
3  }
```

4. 高版本 Android 权限适配（如 Android 13+）

- 动态申请权限：在原生代码中调用 `ActivityCompat.requestPermissions`；
- UniApp 端可先通过 `uni.requestPermission` 申请权限，再调用原生插件。

七、进阶技巧

1. **复杂数据传递**：通过 JSON 序列化（`Gson` / `FastJson`）将对象转为 String 传递，原生侧再反序列化；
2. **生命周期管理**：插件可监听 UniApp 页面生命周期（`onCreate` / `onDestroy`），避免内存泄漏；
3. **性能优化**：耗时操作（如网络请求、硬件扫描）放在子线程执行，完成后通过 `runOnUiThread` 回调主线程；
4. **多插件通信**：通过 `UniSDKInstance` 共享数据，或使用 Android 广播/EventBus。

总结

UniApp Android 原生插件开发的核心是基于 `UniModule` / `UniComponent` 封装原生逻辑，通过 `uniplugin.json` 和 `package.json` 完成配置，最终通过 JSBridge 实现与 UniApp 的交互。开发时需注意**主线程/子线程区分**、**权限适配**、**数据序列化**，调试优先通过原生侧验证逻辑，再集成到 UniApp 中。掌握以上流程后，可轻松扩展 UniApp 对 Android 原生能力的调用。

（注：文档部分内容可能由 AI 生成）